

DevOps Fundamentals

You Must Own

10 essential questions — with answers — covering performance, Kubernetes, infrastructure-as-code, scaling, deployments, incidents, and observability.

Q1

[Performance]

CPU bottleneck vs I/O bottleneck — how to identify each

THE CORE DIFFERENCE

A CPU bottleneck means your processor is the limiting factor — it can't compute fast enough. An I/O bottleneck means your program is spending most of its time waiting: for disk reads, network responses, or database queries. The work is queued, not being done.

CPU BOTTLENECK SIGNALS

- CPU >80% sustained • Low iowait% • Run queue > CPU count • High sys/user time

I/O BOTTLENECK SIGNALS

- CPU iowait% high • Disk await >10ms • CPU looks idle but system is slow • Network: saturation or retransmits

COMMANDS TO RUN

top/htop → watch %wa (iowait). iostat -x 1 → await, %util. vmstat 1 → check b (blocked processes). sar -n DEV 1 → network throughput. If CPU is pegged and iowait is near 0 → CPU bottleneck. If CPU looks idle but iowait is high → I/O bottleneck.

Q2

[Performance]

Why systems fail even when CPU and memory look fine

THE HIDDEN CULPRITS

CPU and memory are only two dials. Production systems fail from resource exhaustion in places that monitoring often misses entirely.

COMMON INVISIBLE FAILURE MODES

- File descriptor limit exhausted • Thread pool saturation • Connection pool full • Open sockets in TIME_WAIT
- Inode exhaustion (disk not full) • Kernel limits: pid/socket/mmap • DNS resolution timeout • GC pauses (JVM) • Lock contention

HOW TO FIND THEM

lsof | wc -l (file descriptors). ss -s (socket states). df -i (inodes). /proc/sys/kernel/pid_max. Check app logs for 'too many open files', 'connection refused', or 'pool exhausted'. These are the first places to look when metrics look healthy but latency spikes.

Q3

[Kubernetes]

What actually happens when a Kubernetes Pod restarts

THE RESTART SEQUENCE

1. Container exits (crash, OOMKilled, or liveness probe failure). 2. kubelet notices via container runtime. 3. Kubelet waits for backoff delay (10s, 20s, 40s... up to 5 min). 4. New container starts — same spec, new process, empty ephemeral storage. 5. Pod IP stays the same. PersistentVolumes remount. In-memory state is gone.

PRESERVED VS LOST

Preserved: Pod IP, PersistentVolume mounts, ConfigMap/Secret mounts, environment variables. Lost: In-memory state, local file system, open connections, in-flight requests.

CRASHLOOPBACKOFF

After repeated failures, Kubernetes enters CrashLoopBackOff — it keeps restarting but with exponential backoff. The pod is alive but useless. 'kubectl logs --previous' shows the last crash's output.

Q4

[Kubernetes]

Readiness vs liveness probes — and how each breaks production

LIVENESS PROBE

Is the container alive? Failure → restart the container. Use for detecting deadlocks or stuck processes.

READINESS PROBE

Is the container ready to serve traffic? Failure → remove from Service endpoints. No restart.

HOW LIVENESS PROBES BREAK PRODUCTION

Set initialDelaySeconds too low and the probe fires before your app finishes startup — Kubernetes restarts it in a loop. Or set the probe to hit a deep endpoint that calls the database: if DB is slow, healthy pods get restarted. Keep liveness probes checking only local liveness (process health), not external dependencies.

HOW READINESS PROBES BREAK PRODUCTION

A too-strict readiness probe removes pods from load balancing during a transient spike, sending all traffic to fewer pods — which then also fail their probes. A cascade. Use generous thresholds (failureThreshold >= 3, periodSeconds >= 10) and check only what is truly required to serve a request.

Q5

[IaC]

What problem does IaC solve, and when does it become dangerous

THE PROBLEM IT SOLVES

Manual infrastructure is slow, error-prone, and undocumented. IaC gives you version-controlled, repeatable, reviewable infrastructure. Every change goes through a PR. Any environment can be recreated from scratch. Drift between environments becomes visible.

WHEN IAC BECOMES DANGEROUS

- Blindly applying without plan review
- Storing secrets in code
- No blast radius limiting (all envs, one apply)
- Destroy/recreate on rename
- Skipping drift detection
- Shared state with no locking

THE HARDEST DANGER

'terraform destroy' is just as easy as 'terraform apply'. A typo in a resource name + no lifecycle protection = deleted production database. Always use deletion protection on stateful resources and require explicit -target on production applies.

Q6

[IaC]

How Terraform state works, and what happens when it drifts

WHAT STATE IS

Terraform maintains a .tfstate file mapping your HCL configuration to real infrastructure resource IDs. It's the source of truth Terraform uses to compute diffs. Without it, Terraform doesn't know what it previously created.

WHAT DRIFT IS

Drift is when reality no longer matches the state file — usually from manual changes in the console, automated scripts, or another tool modifying infrastructure. 'terraform plan' will show these as unexpected changes.

DRIFT CONSEQUENCES

- Plan shows phantom changes
- Apply may overwrite a manual fix
- State corruption if concurrent applies
- Deleted resource still in state

HOW TO HANDLE DRIFT

'terraform refresh' updates state to match reality (read-only). 'terraform import' pulls existing resources into state. Use remote state (S3 + DynamoDB lock) to prevent concurrent modifications. Run drift detection in CI on a schedule — don't find drift during a 3am incident.

Q7

[Scaling]

Horizontal vs vertical scaling — and when to avoid autoscaling

HORIZONTAL (SCALE OUT)

Add more instances. Handles stateless services well. Requires load balancing. Better fault tolerance. Cost scales linearly.

VERTICAL (SCALE UP)

Bigger machine. No code changes needed. Hits hard limits fast. Single point of failure. Good for stateful workloads (databases).

WHEN TO AVOID AUTOSCALING

1. Stateful services without proper session affinity — new pods won't have the user's state. 2. Slow-start workloads where a new pod takes 2+ minutes to warm up — you'll scale too late. 3. When scaling triggers a thundering herd (all new pods hitting the DB at once). 4. If your costs are unpredictable and you have no spend cap.

THE REAL SCALING QUESTION

Before scaling either way, profile first. Most scaling problems are really efficiency problems — N+1 queries, missing indexes, or synchronous calls that could be async.

Q8

[Deployments]

Why deployments succeed but users still see errors

THE GAP

A successful deploy means your new code is running. It says nothing about downstream correctness.

COMMON CAUSES

- CDN caching stale assets
- Feature flag not enabled
- DB migration not applied
- Env var missing in production
- Old pods still serving during rollout
- External API contract changed
- Browser caching old JS
- Smoke tests hitting different host

DEPLOYMENT VS RELEASE

Deployment = code is running in production. Release = users can reach the feature. These are separate events. A deploy can succeed while a release is broken due to config, cache, or routing. Synthetic monitoring (running real user journeys post-deploy) catches this gap.

Q9

[Incidents]

Rollback vs hotfix — how to decide during an incident

ROLLBACK

Revert to the last known-good version. Fast. No new code risk. Loses all changes in the bad deploy including good ones.

HOTFIX

Deploy a targeted fix forward. Requires confident diagnosis. Risks introducing new bugs under pressure.

DECISION FRAMEWORK

Roll back if: you can't identify root cause within 10-15 min, the fix requires significant code changes, or the blast radius is growing. Hotfix if: cause is a config value or a one-line change you can verify in under 5 minutes, and rollback would lose important in-flight state (e.g. a DB migration already applied).

THE DEFAULT SHOULD BE ROLLBACK

Engineers under incident pressure make mistakes. A rollback buys time to diagnose properly. The bias toward hotfixes usually comes from deploy cost anxiety — solve that with faster CI/CD, not incident pressure.

Q10

[Observability]

Metrics, logs, and traces — when to start with each

METRICS

Aggregated numeric measurements over time. Cheap to store. Good for alerting and dashboards. Cannot explain why something is broken.

LOGS

Timestamped events from a specific process. Rich context. Expensive at scale. Good for what happened on a given request.

TRACES

A trace follows a single request across every service it touches, with timing for each hop. Essential for distributed systems. Cannot be reconstructed from logs or metrics alone.

WHEN TO START WITH EACH

Alerting / 'is it broken?' → Metrics first. 'What happened on this request?' → Logs. 'Where is the latency?' / 'Why is p99 high?' → Traces. 'What changed at 14:32?' → Logs.

THE COMMON MISTAKE

Teams instrument metrics and logs but skip traces — then spend hours manually correlating log timestamps across services to reconstruct what a distributed trace would have shown in seconds. If you run more than two services, structured tracing (OpenTelemetry) pays for itself on the first complex incident.

The meta-insight: most DevOps failures aren't caused by the thing you're watching. Systems fail at the boundary between what you measure and what actually matters.